

EE2016: Microprocessor

Experiment 2: Hardware Wiring and Assembly Programming using Atmel Atmega(8) AVR

Name: Shittheeksh H V

Roll No: EE24B063

12th August 2025

Aim:

This experiment introduces assembly programming and interaction with peripherals in Atmel Atmega8 microcontroller.

1. Wire the microcontroller along with the given peripherals in a breadboard, to make it work. For instance, after wiring the peripheral viz, the LED (an output device), write an AVR assembly program to blink an LED. Try another peripheral (input device), the push button and DIP switch.
2. Program the microcontroller to read the DIP switch values and display it in an LED using assembly programming.
3. Program the microcontroller to perform the addition and multiplication of two four-bit numbers which are read from the DIP switches connected to a port and display the result using LED's connected to another port.

Problems:

Problem 1: Write an AVR assembly program to blink an LED with pulse width (during ON) of 0.5 sec and pulse repetition rate of 1 cycle /sec.

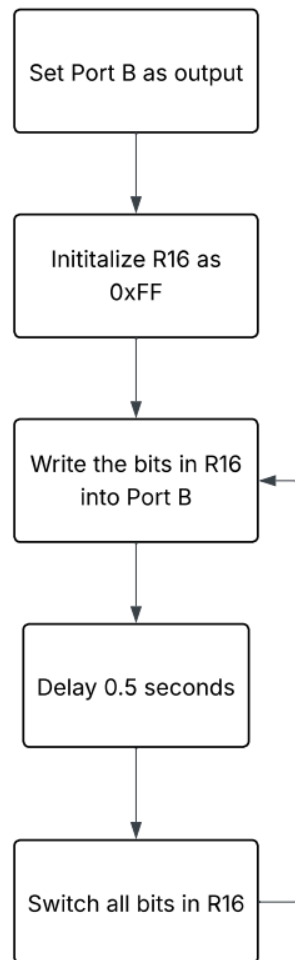
Problem 2: Write an AVR assembly program to control an LED using a push button switch (as long as the button is pushed, the LED should be 'ON').

Problem 3: Implement a 4-bit addition from a single 8-bit GPR register value r1r0 (where r1r0 are the last (LSB) two digits in your roll number) within AVR and load the result of addition into PORTB and see the register values in AVR emulator (Microchip Studio).

Problem 4: 4-bit addition of two unsigned nibbles from an 8-bit DIP input switch and display the result obtained in LEDs.

Problem 1:

Flow Chart:



Code:

```
.INCLUDE "M8DEF.INC"

LDI R16, 0xFF;
OUT DDRB, R16; Setting Port B as output

WRITE:
    OUT PORTB, R16; Writing R16 values into Port B
    RJMP DELAY

SWITCH:
    COM R16; Reverse the bits in R16
    RJMP WRITE

DELAY:
    LDI R25, 2;
D1:
    LDI R26, 200
D2:
    LDI R27, 250
D3:
    NOP; 1 CPU cycle
    NOP; 1 CPU cycle
    DEC R27; 1 CPU cycle
    BRNE D3; 2 CPU cycles (1 cycle in last loop but mostly 2)
    DEC R26; 1 CPU cycle
    BRNE D2; 2 CPU cycles (1 cycle in last loop but mostly 2)
    DEC R25; 1 CPU cycle
    BRNE D1; 2 CPU cycles (1 cycle in last loop but mostly 2)
    RJMP SWITCH

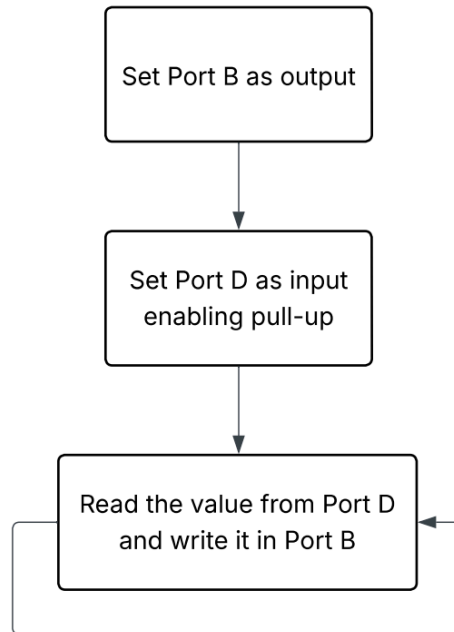
;Loop calculation: (((5*250)+3)*200)+3)*2 = 501206 CPU cycle, each CPU cycle in 1 microsecond
; Time taken= 501206*0.000001 = 0.501206 seconds
```

Output:

https://drive.google.com/file/d/1lorNDViJ6_ghFTQEy1VYNqZ3mvLdFFnU/view?usp=drive_link

Problem 2:

Flow Chart:



Code:

```
.INCLUDE "M8DEF.INC"

LDI R16, 0xFF; Set Port B as output
OUT DDRB, R16

LOOP:
    CBI DDRD, 0
    SBI PORTD, 0; Initiate an internal pull up and also have external pull up resistor

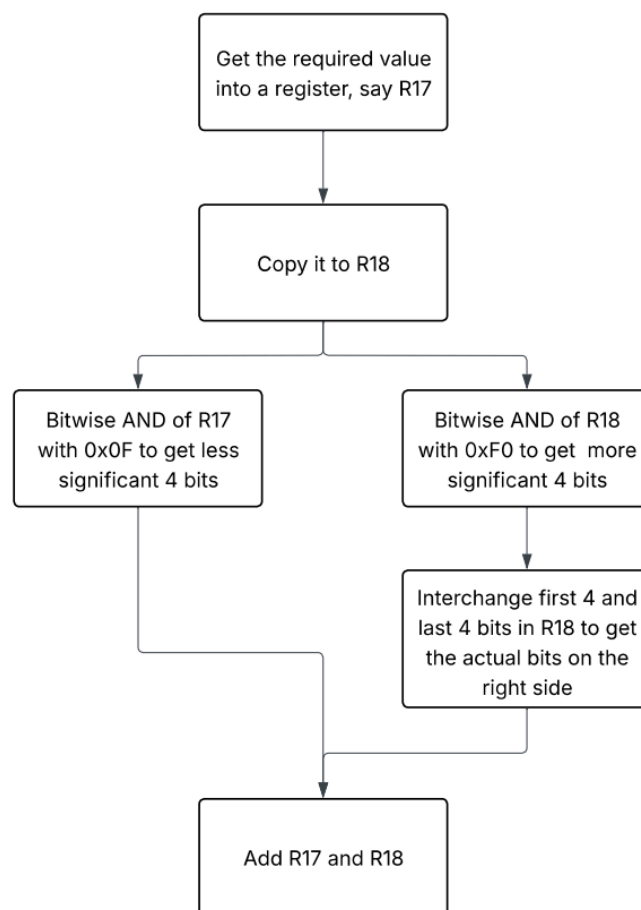
    IN R16, PIND; Read Port D value connected to push button
    OUT PORTB, R16; Output to the LED
    RJMP LOOP
```

Output:

https://drive.google.com/file/d/1kxbx59_hq72wO1Qma5viUacFOoqki-Ju/view?usp=drive_link

Problem 3:

Flow Chart:



Code:

```
.INCLUDE "M8DEF.INC"

LDI R16, 0xFF; Setting Port B as output
OUT DDRB, R16

LDI R17, 0x34; Replace 0x34 with any desired number
MOV R18, R17; R17 and R18 now has the required word which has to be split and added

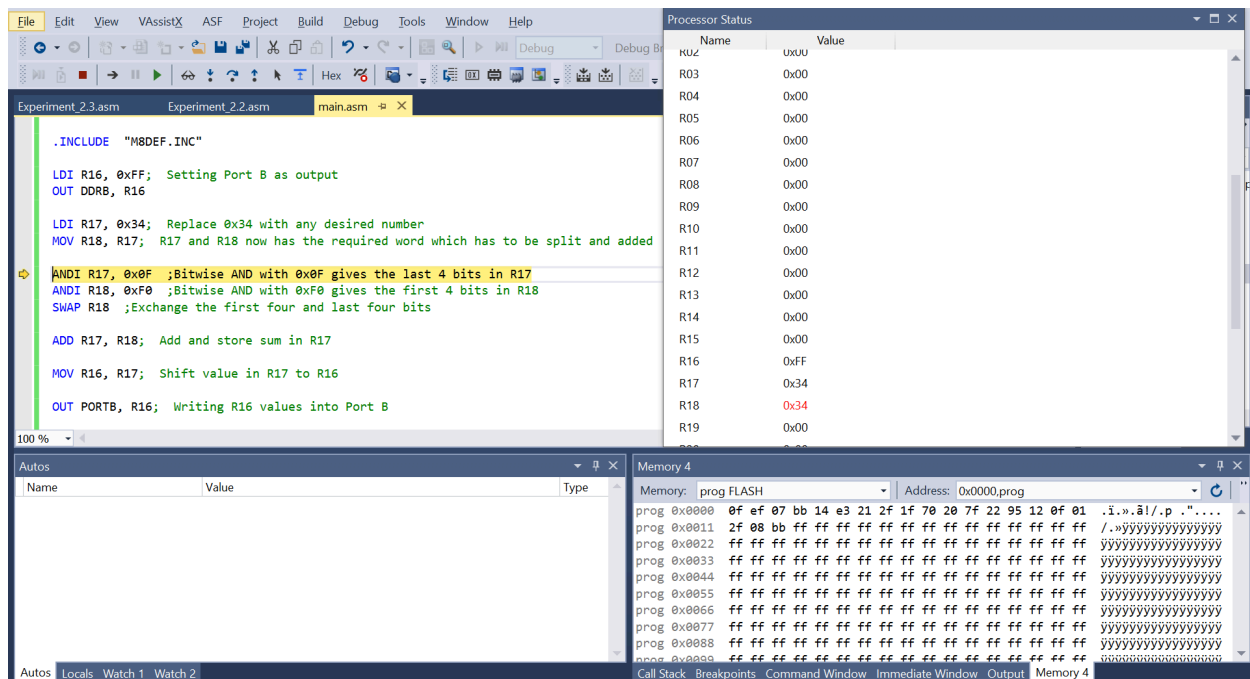
ANDI R17, 0x0F ;Bitwise AND with 0x0F gives the last 4 bits in R17
ANDI R18, 0xF0 ;Bitwise AND with 0xF0 gives the first 4 bits in R18
SWAP R18 ;Exchange the first four and last four bits

ADD R17, R18; Add and store sum in R17

MOV R16, R17; Shift value in R17 to R16

OUT PORTB, R16; Writing R16 values into Port B
```

Output:



File Edit View VAssistX ASF Project Build Debug Tools Window Help

Processor Status

Name	Value
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0xFF
R17	0x04
R18	0x03
R19	0x00

Experiment_2.3.asm Experiment_2.2.asm main.asm

```
.INCLUDE "M8DEF.INC"

LDI R16, 0xFF; Setting Port B as output
OUT DDRB, R16

LDI R17, 0x34; Replace 0x34 with any desired number
MOV R18, R17; R17 and R18 now has the required word which has to be split and added

ANDI R17, 0x0F; Bitwise AND with 0x0F gives the last 4 bits in R17
ANDI R18, 0xF0; Bitwise AND with 0xF0 gives the first 4 bits in R18
SWAP R18; Exchange the first four and last four bits

ADD R17, R18; Add and store sum in R17

MOV R16, R17; Shift value in R17 to R16

OUT PORTB, R16; Writing R16 values into Port B
```

100 %

Autos

Name	Value	Type
------	-------	------

Memory 4

Memory: prog FLASH Address: 0x0000:prog

prog 0x0000	0f ef 07 bb 14 e3 21 2f 1f 70 20 7f 22 95 12 0f 01	.i.».ä!/.p ."....
prog 0x0011	2f 08 bb ff ff ff ff ff ff ff ff ff ff ff ff ff	/.»yyyyyyyyyyyyyyyy
prog 0x0022	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy
prog 0x0033	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy
prog 0x0044	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy
prog 0x0055	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy
prog 0x0066	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy
prog 0x0077	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy
prog 0x0088	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy
prog 0x0099	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy

Autos Locals Watch 1 Watch 2

Call Stack Breakpoints Command Window Immediate Window Output Memory 4

File Edit View VAssistX ASF Project Build Debug Tools Window Help

Processor Status

Name	Value
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x07
R17	0x07
R18	0x03
R19	0x00

Experiment_2.3.asm Experiment_2.2.asm main.asm

```
.INCLUDE "M8DEF.INC"

LDI R16, 0xFF; Setting Port B as output
OUT DDRB, R16

LDI R17, 0x34; Replace 0x34 with any desired number
MOV R18, R17; R17 and R18 now has the required word which has to be split and added

ANDI R17, 0x0F; Bitwise AND with 0x0F gives the last 4 bits in R17
ANDI R18, 0xF0; Bitwise AND with 0xF0 gives the first 4 bits in R18
SWAP R18; Exchange the first four and last four bits

ADD R17, R18; Add and store sum in R17

MOV R16, R17; Shift value in R17 to R16

OUT PORTB, R16; Writing R16 values into Port B
```

100 %

Autos

Name	Value	Type
------	-------	------

Memory 4

Memory: prog FLASH Address: 0x0000:prog

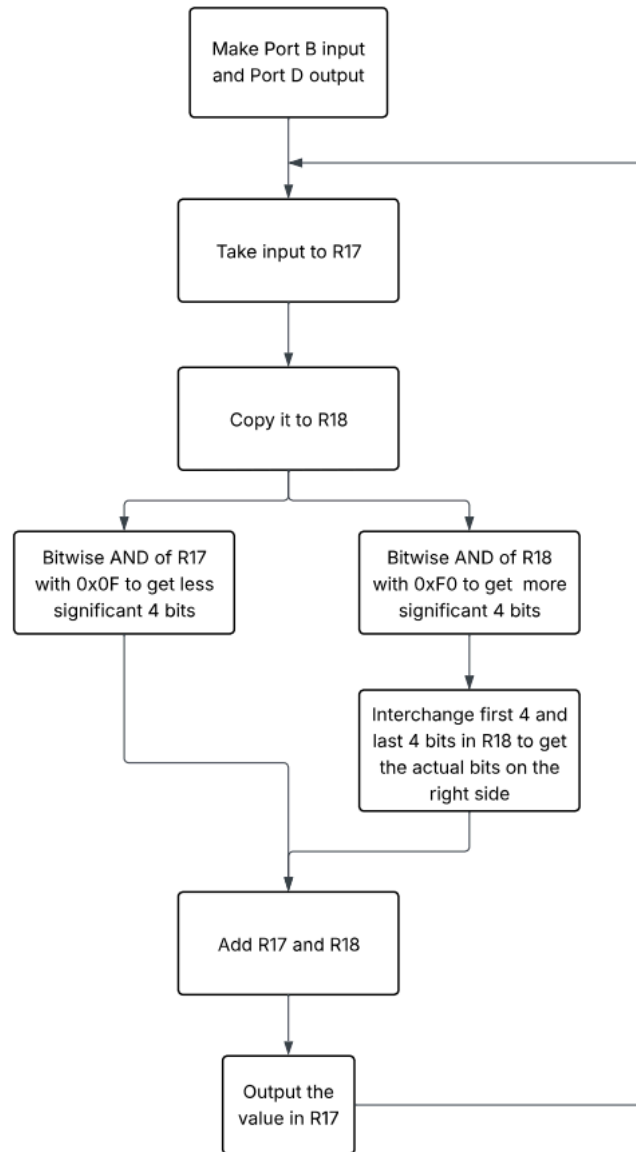
prog 0x0000	0f ef 07 bb 14 e3 21 2f 1f 70 20 7f 22 95 12 0f 01	.i.».ä!/.p ."....
prog 0x0011	2f 08 bb ff ff ff ff ff ff ff ff ff ff ff ff ff	/.»yyyyyyyyyyyyyyyy
prog 0x0022	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy
prog 0x0033	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy
prog 0x0044	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy
prog 0x0055	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy
prog 0x0066	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy
prog 0x0077	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy
prog 0x0088	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy
prog 0x0099	ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff	yyyyyyyyyyyyyyyyyyyy

Autos Locals Watch 1 Watch 2

Call Stack Breakpoints Command Window Immediate Window Output Memory 4

Problem 4:

Flow Chart:



Code:

```
.INCLUDE "M8DEF.INC"

LDI R16, 0x00
OUT DDRB, R16; Make Port B input

COM R16
OUT DDRD, R16; Make Port D output

LOOP:
    OUT PORTB, R16; Enable pull-up

    IN R17, PINB
    MOV R18, R17

    ANDI R17, 0x0F
    ANDI R18, 0xF0

    SWAP R18

    ADD R17, R18; Gives sum after splitting the 8 bits

    OUT PORTD, R17; Output to the LED
    RJMP LOOP
```

Output:

